MASTER AI APP SPECIFICATION & PROMPT

Interactive System Flow Visualizer: Browser to Server

A complete, production-ready AI prompt to build an interactive, animated web application explaining the full journey of a web request through modern backend infrastructure.

1. Overview & Project Objective

This prompt instructs AI coding agents (such as Lovable, Bolt.new, Cursor, v0, or Replit) to generate an interactive, visually stunning, educational visualizer based on the **Complete Request Flow Architecture**. Users can trigger requests, watch live packet animations traverse each infrastructure layer, inspect HTTP payloads/latencies, and simulate branch scenarios (Cache hits, CDN static hits, WAF blocks, and async queue jobs).

2. Master Prompt (Copy & Paste Ready)

Copy the entire prompt block below directly into your AI development tool of choice:

```
You are a Principal Frontend Architect and Systems Engineering Expert. Build a production-grade,
highly polished, interactive web application named:
"RequestFlow: Interactive Browser-to-Server Architecture Simulator"

### 🎯 CORE PURPOSE
Build an interactive educational platform visualizing what happens when a user enters a URL or
submits a form. The app visually traces the entire lifecycle of a network request from the Client
Browser through DNS, Edge CDN, WAF, Load Balancers, API Gateway, Reverse Proxy, Application
Server, Cache, Database, and Asynchronous Message Queue back to the client.

### 🛠️ TECH STACK
- Framework: React 18+ (Vite) / Next.js App Router
- Styling: Tailwind CSS (Dark Mode first: Slate/Zinc 950 backdrop, vibrant cyan/emerald/violet
accents)
- Animation: Framer Motion (smooth packet traversal, pulsating nodes, glowing beam paths)
- Icons: Lucide React
- Components & State: Radix UI / Shadcn UI primitives, Zustand or React Context for simulation
state
- Layout: Responsive desktop canvas with full zoom/pan support & mobile-friendly step-by-step
stepper

### 🗺️ ARCHITECTURAL NODES TO IMPLEMENT (Sequential Flow)
1. Browser (User): Initiates HTTP/HTTPS GET, POST, or Static Asset requests. Displays URL bar &
Send button.
2. DNS Resolution: Simulates Local Cache & Recursive DNS lookup resolving domain name to IP (e.g.,
104.21.32.1).
3. Internet / ISP Gateway: Demonstrates routing across public WAN with simulated packet hops &
latency.
4. Firewall (Edge Security / WAF): Cloudflare/AWS WAF layer inspecting traffic. Can drop malicious
payloads (DDoS/SQLi).
5. CDN Edge Node: CloudFront/Fastly edge server. Resolves static assets (CSS, JS, WebP) with near
0ms latency.
6. Load Balancer: Layer 7 AWS ALB / HAProxy distributing requests via Round Robin or Least
Connections to healthy clusters.
7. API Gateway: Kong/Envoy handling JWT auth token validation, CORS headers, and rate-limiting
quotas.
8. Reverse Proxy (NGINX): Handles SSL/TLS termination, gzip/brotli compression, and reverse
routing to private subnet app servers.
9. Application Server: Node.js/Go/Python backend executing business logic, controller methods, and
```

```
data parsing.
10. Cache Layer (Redis): In-memory key-value store. Checks for cache HIT (returns in 1-2ms) or cache MISS.
11. Primary Database: PostgreSQL/MySQL transactional storage executing ACID queries and indexing.
12. Message Queue (Kafka/RabbitMQ): Decoupled asynchronous worker queue for background processing (emails, analytics).
13. Response Route: Formats JSON/HTML payload and routes back through the established tunnel to Browser render.

### 🎮 INTERACTIVE SIMULATION MODES
Provide a scenario selector toolbar:
- Scenario A: "Standard API Read" (Browser -> DNS -> WAF -> LB -> API Gateway -> Nginx -> App -> Redis HIT -> Response)
- Scenario B: "Heavy Database Write" (Browser -> Auth -> App Server -> PostgreSQL Write -> Kafka Async Publish -> 201 Created)
- Scenario C: "Static Asset CDN Hit" (Browser -> DNS -> CDN Cache HIT -> Immediate 200 OK without hitting backend servers)
- Scenario D: "Security Threat Blocked" (Browser -> WAF detects SQL Injection payload -> 403 Forbidden immediate drop)
- Scenario E: "Step-by-Step Guided Tour" (Allows user to step forward/backward with explanations at each station)

### 🖥️ USER INTERFACE & PANELS
1. Top Interactive Control Bar:
   - Request Type Selector (GET /api/user, POST /api/checkout, GET /styles.css)
   - Play, Pause, Reset, and Speed Slider (0.5x, 1x, 2x)
   - Scenario dropdown presets
2. Central Architecture Canvas:
   - SVG-powered node network with glowing animated packets (dots traveling along path lines).
   - Each node displays: Icon, Component Name, Status Indicator (Idle, Processing, Success, Blocked), and Live Latency badge (e.g. "12ms").
3. Side Inspector Drawer / Modal (Opens on node click):
   - "Why this component exists" breakdown
   - Real-world software equivalents (e.g. NGINX, Redis, Cloudflare, Kafka)
   - Live HTTP Headers & Payload inspector (Request Headers, Query Params, Response Codes)
   - Latency contribution breakdown (e.g. Total Roundtrip: 142ms)
4. Bottom Architecture Cheat Sheet:
   - Summary cards detailing each component's key role, security benefits, and scalability impact.

### ✨ VISUAL DESIGN REQUIREMENTS
- Futuristic, developer-centric theme (Deep navy `#090d16` background with glassmorphic cards).
- Distinct color-coded packets: Neon Cyan for HTTP GET, Emerald Green for 200 OK Responses, Amber for Queue Jobs, Crimson for 403/500 Errors.
- Responsive layout with zero horizontal bleed; smooth transitions between active node highlights.
```

3. Infrastructure Component Matrix

Component	Real-World Tech	Core Responsibility in the Pipeline	Typical Latency
1. Browser	Chrome / Safari	Renders DOM, caches resources locally, initiates HTTP/3 or HTTPS TCP handshake.	0 ms (Origin)
2. DNS Resolution	Route 53 / 1.1.1.1	Resolves human domain name (e.g., example.com) to target Anycast IP address.	10 – 45 ms
3. Internet / ISP	BGP Routing	Transmits IP packets through optic fiber backbones, ISPs, and regional exchanges.	15 – 80 ms
4. Firewall (WAF)	Cloudflare WAF / AWS		2 – 8 ms

Component	Real-World Tech	Core Responsibility in the Pipeline	Typical Latency
		Filters malicious requests, defends against Layer 7 DDoS, SQLi, and bot abuse.	
5. CDN Edge Node	CloudFront / Fastly	Delivers cached static assets (images, CSS, JS) from edge server nearest to the user.	5 – 25 ms
6. Load Balancer	AWS ALB / HAProxy	Distributes incoming traffic across autoscaling app clusters to avoid bottlenecks.	2 – 5 ms
7. API Gateway	Kong / Apigee / Envoy	Enforces rate limiting, validates JWT tokens, handles API versioning and telemetry.	3 – 10 ms
8. Reverse Proxy	NGINX / Caddy	Terminates SSL/TLS certificates, balances local sockets, and compresses responses.	1 – 4 ms
9. Application Server	Node.js / Go / Django	Executes core business logic, input validation, and inter-service communications.	15 – 60 ms
10. Cache Layer	Redis / Memcached	Stores high-frequency hot data in RAM for sub-millisecond retrieval.	1 – 3 ms
11. Primary Database	PostgreSQL / MySQL	Relational durable persistence storage with ACID transaction compliance.	10 – 50 ms
12. Message Queue	Kafka / RabbitMQ	Buffers asynchronous, non-blocking background tasks (push notifications, ingestion).	Async / Decoupled

4. Key Interactive Features to Highlight to the AI

🚀 **Interactive Micro-Interactions & Gamification**

- **Packet Path Glowing Effect:** When a request starts, draw a luminous line passing from node to node with dynamic SVG stroke dash-offset animation.
- **Node Telemetry HUD:** Hovering or tapping any node pops up a micro-telemetry card showing CPU load, active connections, and cache hit/miss ratio.
- **Interactive HTTP Inspector:** Displays real mock headers (e.g. `Host`, `Authorization: Bearer ...`, `CF-Ray`, `X-Cache: HIT`, `Content-Type: application/json`).
- **Failure Injection Mode:** Add toggle switches to let users simulate a database outage (500 Error) or Redis cache eviction to see how fallback mechanics work.

💡 **How to Use:** Copy the text in Section 2 directly into **Lovable.dev**, **Bolt.new**, **Cursor**, or **v0.dev**. The prompt includes all state definitions, node structures, and styling guidelines required to generate the complete application in a single generation session.